

Cours Java

Arrays, Strings, List & Java Collections Framework

Les structures de donnees de base en Java :
tableaux, chaines de caracteres, listes, et la hierarchie des interfaces
du Collections Framework.

Table des matières

1	Arrays (tableaux) en Java	2
1.1	Parcours d'un tableau	2
1.2	La classe utilitaire <code>Arrays</code>	2
1.3	Convertir entre tableau et liste	3
2	Strings (chaines de caracteres) en Java	4
2.1	Methodes essentielles	4
2.2	Construire une chaine efficacement : <code>StringBuilder</code>	4
2.3	Manipuler les caracteres	5
3	List en Java	6
3.1	<code>ArrayList</code> vs <code>LinkedList</code>	6
3.2	Parcourir et trier une liste	7
4	Java Collections Framework : interfaces et hierarchie	8
4.1	Vue d'ensemble de la hierarchie	8
4.2	L'interface <code>Collection<E></code>	8
4.3	<code>List<E></code> : sequence ordonnee, doublons autorises	8
4.4	<code>Set<E></code> : elements uniques, pas de doublons	8
4.5	<code>Queue<E></code> et <code>Deque<E></code>	9
4.6	Comparaison synthetique des trois familles	9

1. Arrays (tableaux) en Java

Un tableau (**array**) est la structure de données la plus basique de Java : une zone mémoire **contigue** de **taille fixe**, dont tous les éléments ont le **même type**. C'est le bloc de construction sur lequel sont bâtis `ArrayList`, `String` (en interne, un tableau de `char/byte`), et une bonne partie du Collections Framework.

L'idée. Parce que la taille est fixée à la création et que les éléments sont contigus en mémoire, l'accès à l'indice `i` se fait par un simple calcul d'adresse (`base + i * taille_element`) : c'est pour cela que l'accès par indice est en $O(1)$. En contrepartie, on ne peut ni agrandir ni retrecir un tableau après sa création.

Declaration et initialisation

```
int[] nums = new int[5];           // tableau de 5 entiers, initialise à 0
int[] nums2 = {1, 2, 3, 4, 5};     // initialisation directe
int[] nums3 = new int[]{1, 2, 3};  // forme explicite équivalente

int[][] matrix = new int[3][4];    // tableau 2D : 3 lignes, 4 colonnes
int[][] grid = {{1, 2}, {3, 4}};   // initialisation directe d'une matrice

System.out.println(nums.length);   // longueur : propriété, PAS une méthode
```

Piège classique

`nums.length` est un **champ**, pas une méthode : pas de parenthèses. À l'inverse, pour une `String`, c'est `str.length()` avec parenthèses (une méthode). Confondre les deux est l'erreur de syntaxe la plus fréquente chez les débutants en Java venant d'autres langages.

Parcours d'un tableau

```
int[] nums = {10, 20, 30, 40};

// Boucle indexée classique
for (int i = 0; i < nums.length; i++) {
    System.out.println(nums[i]);
}

// Boucle "for-each" (enhanced for) : plus lisible si on n'a pas besoin de l'indice
for (int n : nums) {
    System.out.println(n);
}
```

Reconnaitre quand utiliser quelle boucle. Utilise le **for-each** dès que tu n'as besoin que de la *valeur*. Reviens à la boucle indexée dès que tu as besoin de l'**indice** lui-même (modifier le tableau en place, comparer des positions, parcourir à l'envers, ou avancer de plusieurs pas à la fois).

La classe utilitaire Arrays

Le tableau lui-même n'a presque aucune méthode ; toutes les opérations utiles passent par la classe statique `java.util.Arrays`.

```
import java.util.Arrays;

int[] nums = {5, 3, 8, 1};
```

```
Arrays.sort(nums); // tri en place, O(n log n)
System.out.println(Arrays.toString(nums)); // affichage lisible : [1, 3, 5, 8]

int[] copie = Arrays.copyOf(nums, 6); // copie avec nouvelle taille (padding a 0)
int[] sousTab = Arrays.copyOfRange(nums, 1, 3); // sous-tableau [indice 1, 3[

boolean egaux = Arrays.equals(nums, copie); // comparaison element par element
Arrays.fill(nums, 0); // remplit tout le tableau avec 0

// binarySearch exige un tableau DEJA TRIÉ, sinon resultat indefini
int idx = Arrays.binarySearch(nums, 5);
```

Complexite : Temps $O(1)$ acces par indice ; $O(n)$ recherche lineaire ; $O(n \log n)$ tri
| Espace $O(n)$

Piege classique

Un tableau d'objets (`int[][]`, `Object[]`) n'a **pas** de `toString()` lisible par default : `System.out.println(nums)` affiche une adresse memoire du type `[I@1b6d3586`. Il faut toujours passer par `Arrays.toString(nums)` (tableau 1D) ou `Arrays.deepToString(matrix)` (tableau 2D).

Convertir entre tableau et liste

C'est une source frequente de bugs subtils en entretien.

```
Integer[] boxed = {1, 2, 3};
List<Integer> vue = Arrays.asList(boxed); // VUE sur le tableau, taille fixe !
// vue.add(4); // UnsupportedOperationException : on ne peut pas l'agrandir

List<Integer> vraieListe = new ArrayList<>(Arrays.asList(boxed)); // copie modifiable

// Attention : Arrays.asList ne fonctionne pas comme attendu sur un tableau primitif
int[] primitifs = {1, 2, 3};
List<int[]> mauvais = Arrays.asList(primitifs); // liste d'UN SEUL element (le tableau
// lui-meme) !
```

Piege classique

`Arrays.asList(tableauPrimitif)` sur un `int[]` ne cree pas une `List<Integer>` de 3 elements, mais une `List<int[]>` d'un seul element (a cause de l'autoboxing qui ne s'applique qu'aux types objets). Il faut un tableau de `Integer[]` (type objet) pour que la conversion se comporte comme attendu, ou utiliser `Arrays.stream(primitifs).boxed()`.

2. Strings (chaines de caracteres) en Java

En Java, `String` est un **objet immuable** : une fois cree, son contenu ne peut jamais changer. Toute operation qui "modifie" une chaine (+, `substring`, `replace`...) cree en realite une **nouvelle** chaine et laisse l'originale intacte.

L’idee. L’immuabilite permet a la JVM de reutiliser en toute securite la meme instance pour deux litteraux identiques (le *String Pool*), et rend les `String` intrinsequement thread-safe. Le prix a payer : creer une chaine dans une boucle avec des concatenations += recree un nouvel objet a chaque iteration, ce qui devient couteux en $O(n^2)$ sur n concatenations.

Quand l'utiliser

Toute donnee textuelle immuable : cles de map, comparaisons, parsing. Des que tu dois construire une chaine **progressivement** (boucle, concatenations repetees), pense `StringBuilder` plutot que `String`.

Methodes essentielles

```
String s = "Hello, World!";

s.length();           // 13 -- methode, PAS un champ (contrairement a un tableau)
s.charAt(0);           // 'H'
s.substring(7);        // "World!"      (a partir de l'indice 7)
s.substring(7, 12);    // "World"      (indice 7 inclus, 12 exclu)
s.indexOf("World");    // 7            (-1 si non trouve)
s.toLowerCase();       // "hello, world!"
s.toUpperCase();       // "HELLO, WORLD!"
s.trim();              // supprime les espaces en debut/fin
s.strip();             // equivalent moderne (Java 11+), gere aussi l'Unicode
s.replace("World", "Java"); // "Hello, Java!"
s.split(", ");         // String[] {"Hello", "World!"}
s.contains("World");   // true
s.startsWith("Hello"); // true
s.equals("hello, world!"); // false -- sensible a la casse
s.equalsIgnoreCase("HELLO, WORLD!"); // true

String.join("-", "a", "b", "c"); // "a-b-c"
"".isBlank();              // true (Java 11+)
"".isEmpty();              // true
```

Piege classique

Ne **jamais** comparer deux `String` avec `==` : cela compare des references memoire, pas le contenu. `new String("abc") == "abc"` vaut `false`. Toujours utiliser `.equals()` (ou `.equalsIgnoreCase()`).

Construire une chaine efficacement : `StringBuilder`

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) {
    sb.append(i).append(","); // append est chainable
}
sb.deleteCharAt(sb.length() - 1); // retire la derniere virgule
String resultat = sb.toString(); // "0,1,2,3,4"

sb.reverse(); // inverse le contenu en place
```

```
sb.insert(0, "prefix-");           // insertion a un indice donne
```

Complexite : Temps $O(1)$ amorti par append | Espace $O(n)$

Reconnaitre le pattern

Des que l'enonce implique de construire une chaine *caractere par caractere* ou *morceau par morceau* dans une boucle (inverser une chaine, construire un resultat progressif, palindromes), utilise **StringBuilder** : cela transforme un cout quadratique en cout lineaire.

Manipuler les caracteres

```
char c = 'a';
Character.isDigit(c);           // false
Character.isLetter(c);          // true
Character.isUpperCase(c);       // false
Character.toUpperCase(c);        // 'A'

// Convertir String en tableau de char pour un acces/tri caractere par caractere
char[] chars = "hello".toCharArray();
Arrays.sort(chars);              // tri des caracteres
String trie = new String(chars); // reconversion en String
```

Problemes types : Valid Anagram (LC 242), Reverse String (LC 344), Longest Common Prefix (LC 14), Valid Palindrome (LC 125).

3. List en Java

`List<E>` est une **interface** du Collections Framework représentant une sequence ordonnee d'elements, avec doublons autorises et acces par indice. Elle resout la principale limite des tableaux : la **taille fixe**.

L'idée. On programme presque toujours contre l'interface `List<E>` plutot que contre une implementation concrete (`ArrayList`, `LinkedList`) : cela permet de changer d'implementation plus tard sans modifier le reste du code, et c'est la convention Java standard (« coder contre une interface »).

```
List<Integer> list = new ArrayList<>();    // le type declare est l'interface,
                                           // le type instancie est l'implementation

list.add(10);
list.add(20);
list.add(0, 5);                          // insertion a l'indice 0 : [5, 10, 20]
list.get(1);                             // 10
list.set(1, 99);                          // remplace l'element a l'indice 1 : [5, 99, 20]
list.remove(0);                           // remove(int) : supprime PAR INDICE -> [99, 20]
list.remove(Integer.valueOf(99));          // remove(Object) : supprime PAR VALEUR -> [20]
list.contains(20);                        // true
list.indexOf(20);                         // 0
list.size();                             // 1
list.isEmpty();                           // false
list.clear();                             // vide la liste

List<Integer> literal = List.of(1, 2, 3); // liste IMMUTABLE (Java 9+)
```

Piege classique

Sur une `List<Integer>`, `list.remove(0)` et `list.remove(Integer.valueOf(0))` **ne font pas la meme chose** : la premiere supprime l'element a l'indice 0 (surcharge `remove(int)`), la seconde supprime la *valeur* 0 (surcharge `remove(Object)`). C'est une ambiguite classique qui piege beaucoup de candidats en entretien.

ArrayList vs LinkedList

Les deux implementent `List<E>`, mais avec des structures internes tres differentes.

Operation	ArrayList (tableau redimensionnable)	LinkedList (liste doublement chainee)
Acces par indice <code>get(i)</code>	$O(1)$	$O(n)$
Ajout/suppression en fin	$O(1)$ amorti	$O(1)$
Ajout/suppression en debut	$O(n)$ (decalage)	$O(1)$
Ajout/suppression au milieu	$O(n)$ (decalage)	$O(n)$ pour trouver + $O(1)$ pour inserer
Empreinte memoire	Compacte	Plus lourde (pointeurs prev/next par noeud)

Reconnaitre quand utiliser quoi

`ArrayList` est le choix par défaut dans l'immense majorité des cas : les accès par indice et les parcours dominent en pratique. Ne choisis `LinkedList` que si tu fais *beaucoup* d'insertions/suppressions en tête ou en queue (elle implémente aussi `Deque<E>`, utile pour une pile ou une file).

```
LinkedList<Integer> deque = new LinkedList<>();
deque.addFirst(1);    // ajout en tête, O(1)
deque.addLast(2);     // ajout en queue, O(1)
deque.removeFirst();  // retrait en tête, O(1)
deque.peekLast();     // consultation sans retrait
```

Parcourir et trier une liste

```
List<Integer> nums = new ArrayList<>(List.of(5, 3, 8, 1));

for (int n : nums) {           // for-each
    System.out.println(n);
}

Collections.sort(nums);        // tri croissant en place, O(n log n)
nums.sort(Comparator.reverseOrder()); // tri décroissant, méthode d'instance (Java 8+)

// Tri avec comparateur personnalisé (ex : liste de String par longueur)
List<String> mots = new ArrayList<>(List.of("banane", "kiwi", "pomme"));
mots.sort((a, b) -> a.length() - b.length());
mots.sort(Comparator.comparingInt(String::length)); // équivalent, plus idiomatique
```

Piège classique

Modifier une liste (add/remove) **pendant** un `for-each` lève une `ConcurrentModificationException`. Pour supprimer des éléments en parcourant, utilise un `Iterator` explicite avec `iterator.remove()`, ou `list.removeIf(condition)`.

```
// Suppression sûre pendant un parcours
Iterator<Integer> it = nums.iterator();
while (it.hasNext()) {
    if (it.next() % 2 == 0) {
        it.remove();    // seule façon sûre de supprimer pendant l'iteration
    }
}

// Équivalent moderne et plus lisible (Java 8+)
nums.removeIf(n -> n % 2 == 0);
```

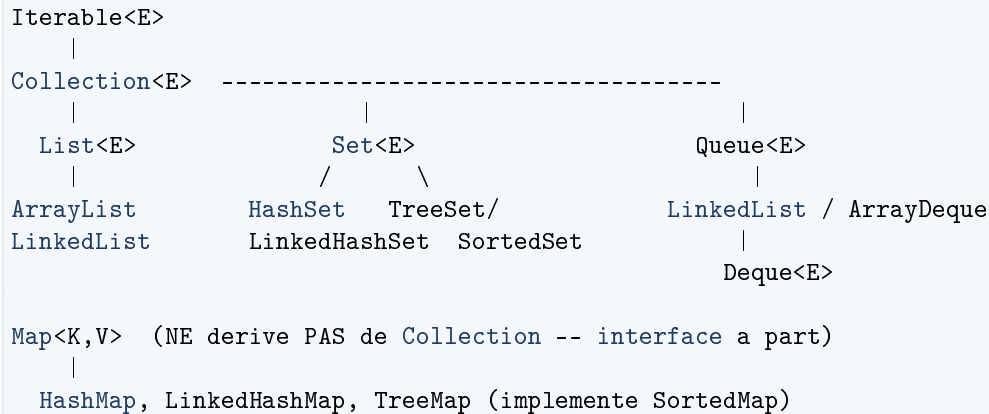
Complexité : Temps Voir tableau ci-dessus selon l'implémentation | Espace $O(n)$

Problèmes types : Rotate Array (LC 189), Remove Duplicates from Sorted Array (LC 26), Merge Sorted Array (LC 88).

4. Java Collections Framework : interfaces et hierarchie

Le Collections Framework est l'ensemble unifié d'interfaces et de classes de `java.util` pour manipuler des groupes d'objets. Comprendre sa **hierarchie d'interfaces** est essentiel : en entretien comme en production, on déclare toujours des variables avec le type **interface** le plus général possible (`List`, `Map`, `Set`), jamais avec le type d'implémentation concrète.

Vue d'ensemble de la hiérarchie



Point souvent testé en entretien

`Map<K,V>` ne fait **pas** partie de la hiérarchie `Collection<E>` : c'est une interface racine indépendante, car une map manipule des *paires* cle-valeur et non de simples éléments. C'est une question de compréhension fréquemment posée pour vérifier qu'on connaît la hiérarchie plutôt que de l'avoir simplement mémorisée par cœur.

L'interface `Collection<E>`

Racine commune de `List`, `Set` et `Queue`. Elle définit le contrat partagé par toutes les collections d'éléments uniques (au sens "un seul type d'élément", doublons ou non selon la sous-interface) : `add`, `remove`, `contains`, `size`, `isEmpty`, `iterator`, `clear`, `stream`.

`List<E>` : séquence ordonnée, doublons autorisés

Déjà détaillée section précédente. Implémentations : `ArrayList` (tableau redimensionnable), `LinkedList` (liste chaînée).

`Set<E>` : éléments uniques, pas de doublons

Implementation	Structure interne	Caractéristique
HashSet	Table de hachage (adosée à une <code>HashMap</code>)	Pas d'ordre garanti ; $O(1)$ moyen pour <code>add/contains</code>
LinkedHashSet	Table de hachage + liste chaînée	Conserve l'ordre d'insertion ; $O(1)$ moyen
TreeSet	Arbre rouge-noir	Éléments toujours triés ; $O(\log n)$ pour <code>add/contains</code>

```
Set<Integer> set = new HashSet<>();
set.add(5);
set.add(5);           // ignore silencieusement, deja present
set.contains(5);      // true, O(1) moyen

Set<Integer> ordonne = new TreeSet<>(Set.of(5, 1, 3)); // {1, 3, 5} -- toujours trie
```

Reconnaitre le pattern

Des que l'enonce parle d'*unicite*, de *doublons a supprimer*, ou de tester une appartenance repetee dans une boucle (if (list.contains(x)) en $O(n)$ a chaque appel), remplace la structure par un **Set** : le test d'appartenance passe de $O(n)$ a $O(1)$ moyen.

Queue<E> et Deque<E>

Queue<E> modelise une file (FIFO – premier entre, premier sorti). **Deque<E>** (*double-ended queue*) generalise a une file a deux extremités, et peut donc aussi servir de **pile** (LIFO).

```
Queue<Integer> file = new LinkedList<>();
file.offer(1);       // ajoute en fin (equivalent add, mais ne leve pas d'exception
                    // si echec)
file.poll();         // retire et retourne la tete (null si vide, contrairement a
                    // remove())
file.peek();         // consulte la tete sans retirer

Deque<Integer> pile = new ArrayDeque<>(); // ArrayDeque : implementation recommandee
                    // pour une pile
pile.push(1);        // empile
pile.push(2);
pile.pop();           // depile -> 2 (LIFO)
pile.peek();         // consulte le sommet -> 1
```

Piege classique

La classe historique `java.util.Stack` existe toujours mais est **deconseillee** : elle herite de **Vector**, une classe synchronisee (donc plus lente inutilement) et mal concue. Utiliser **ArrayDeque** pour implementer une pile en Java moderne.

Comparaison synthetique des trois familles

Interface	Doublons ?	Ordre ?	Cas d'usage typique
List<E>	Oui	Ordre d'insertion, indexe	Sequence, historique, resultat ordonne
Set<E>	Non	Depend de l'implementa- tion	Unicite, appartenance rapide
Queue/Deque	Oui	FIFO ou LIFO	File d'attente, pile, BFS/DFS
Map<K, V>	Cles non, valeurs oui	Depend de l'implementa- tion	Association cle → valeur, comptage, cache

A retenir

Toujours **declarer le type interface** et **instancier l'implementation** : `List<Integer> l = new ArrayList<>();` plutot que `ArrayList<Integer> l = new ArrayList<>();`. Cela decouple le code du choix d'implementation et permet de le changer plus tard (par ex. passer d'`ArrayList` a `LinkedList`) sans toucher au reste du programme.

Problemes types : Design HashSet (LC 705), Implement Queue using Stacks (LC 232), Min Stack (LC 155).